Unit - 3

JavaScript Control Statements

Control statements in JavaScript allow you to control the flow of program execution based on conditions or repetitions. The main types are:

1. Conditional Statements

if, else if, else

These statements let your program make decisions.

Syntax & Example:

```
let temperature = 25;
if (temperature > 30) {
  console.log("It's hot!");
} else if (temperature >= 20) {
  console.log("It's warm.");
} else {
  console.log("It's cold.");
}
```

Explanation:

- If temperature > 30 — prints "It's hot!"
- If temperature between 20 and 30 — prints "It's warm."
- Otherwise — prints "It's cold."

switch Statement

Useful when testing one variable against different possible values.

Syntax & Example:

```
let color = "green";
switch (color) {
  case "red":
    console.log("Stop");
    break;
  case "green":
    console.log("Go");
    break;
  case "yellow":
    console.log("Caution");
    break;
  default:
    console.log("Unknown color");
}
```

Explanation:

- Checks each case and executes the matching block.
- `break` prevents falling through to other cases.
- `default` runs if no case matches.

2. Looping Statements

They help you repeat code multiple times.

for Loop

Used for repeating code a specific number of times.

Syntax & Example:

```
for (let i = 1; i <= 5; i++) {
  console.log("Count: " + i);
```

```
}
```

Explanation:

- Runs five times, printing "Count: 1" through "Count: 5".

while Loop

Repeats as long as a condition is true.

Syntax & Example:

```
let count = 1;
while (count <= 3) {
  console.log("Number: " + count);
  count++;
}
```

Explanation:

- Prints 1, 2, 3.

do...while Loop

Same as `while`, but always runs at least once.

Syntax & Example:

```
let number = 5;
do {
  console.log("Value: " + number);
  number--;
} while (number > 0);
```

Explanation:

- Prints from 5 down to 1.

3. Jump Statements

Used for breaking or continuing loops.

break Statement

Exits from a loop or switch statement.

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) break; // Exit loop when i is 5
  console.log(i);
}
// Prints 1 to 4
```

continue Statement

Skips the current loop iteration and continues with the next.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue; // Skip when i is 3
  console.log(i);
}
// Prints 1, 2, 4, 5 (skips 3)
```

Summary Table

Control Statement	Purpose	Example use case
if / else if / else	Branching logic	Grading, login checks
switch	Multiple fixed values	Menu selection, traffic light
for	Counted repetitions	Printing numbers, loops
while	Conditional repetitions	Waiting for input
do...while	At least once repetition	Displaying menu at least once

break	Exit loop/switch early	Find/search exit
continue	Skip rest of current loop iteration	Skip certain items

Key Points:

- Control statements make your programs intelligent and interactive.
- Use if/else for two or more choices; switch for multiple fixed options.
- Loop statements reduce repetition and make code cleaner.
- break and continue manage flow inside loops.

Object Creation and Modification

Objects are central to JavaScript. They are collections of properties—key/value pairs—that can hold any data, including other objects and functions.

1. Object Creation

a. Object Literal

The simplest and most common way to create objects.

```
let person = {
  name: "Alice",
  age: 25,
  isStudent: true
};
```

Explanation:

This creates an object named `person` with three properties: `name`, `age`, and `isStudent`.

b. Using the `new Object()` Constructor

```
let car = new Object();
car.brand = "Toyota";
car.year = 2020;
```

Explanation:

First creates an empty object, then adds properties using dot notation.

c. Constructor Function

Create multiple similar objects using functions.

```
function Animal(type, sound) {
  this.type = type;
  this.sound = sound;
}
let cat = new Animal("Cat", "Meow");
let dog = new Animal("Dog", "Bark");
```

Explanation:

The `Animal` function acts as a blueprint for new animal objects.

d. Object.create() Method

Create a new object with a specified prototype.

```
let parent = { greeting: "hello" };
let child = Object.create(parent);
child.name = "Sam";
console.log(child.greeting); // Output: hello
```

Explanation:

The `child` object inherits from `parent`.

2. Accessing and Modifying Object Properties

a. Access Properties

- Dot notation: `object.property`
- Bracket notation: `object["property"]` (useful for dynamic or multi-word keys)

```
let student = { name: "Ryan", grade: "A" };
console.log(student.name);     // Output: Ryan
console.log(student["grade"]); // Output: A
```

b. Add or Update Properties

Simply assign a value to a property:

```
student.age = 18;        // Adds a new property
student.grade = "A+";    // Updates existing property
```

Explanation:

If the property doesn't exist, it is added. If it does exist, value is updated.

c. Delete Properties

Use the `delete` keyword to remove a property.

```
delete student.age; // Removes the 'age' property
```

d. Check If Property Exists

Use `in` operator or `hasOwnProperty()`:

```
console.log("name" in student);          // true
console.log(student.hasOwnProperty("age"));// false (if deleted above)
```

3. Iterating Over Object Properties

Use for...in loop to go through all properties.

```
for (let key in student) {
  console.log(key + ": " + student[key]);
}
```

4. Nested Objects

Objects can have properties that are themselves objects.

```
let company = {
  name: "TechCorp",
  address: {
    city: "New York",
    zip: "10001"
  }
};
console.log(company.address.city); // Output: New York
```

5. Objects and Functions

Objects can also contain functions (methods):

```
let user = {
  name: "Sara",
  greet: function() {
    console.log("Hello, " + this.name);
  }
};
user.greet(); // Output: Hello, Sara
```

Summary Table

Operation	Syntax	Example

Create	{} or new Object()	let obj = {};
Access property	obj.key	obj.name
Add/Update	obj.key = value	obj.age = 20;
Delete	delete obj.key	delete obj.name;
Check existence	"key" in obj	"age" in obj

Key Points:

- Objects store data as key/value pairs.
- You can freely add, update, and delete properties.
- Objects can include other objects or functions.

javascript Arrays & Functions

JavaScript arrays are special objects that store ordered lists of data, and **functions** are blocks of reusable code. Arrays and functions are fundamental concepts in JavaScript and often work together for processing data efficiently.

JavaScript Arrays:

- Declared using square brackets [ ] or the Array constructor.

```
const numbers = [1, 2, 3];
const fruits = new Array("Apple", "Banana")[1][3][7][9][16];
```

- Array elements are accessed via zero-based indices: numbers gives 1.
- The .length property returns the number of elements: numbers.length // 3.[1][2][3]
- You can add items with push() (end), unshift() (start), or by assigning to array[index].
- Remove items with pop() (end), shift() (start), or splice().
- Arrays can store items of any data type—even functions or other arrays.

Example: Iterating Through an Array

```
const animals = ["cat", "dog", "fish"];
animals.forEach(function(animal) {
  console.log(animal);
});
// Output: cat dog fish
[^3][^4]
```

JavaScript Functions:

- Defined with the `function` keyword or as arrow functions.

  ```
  function greet(name) {
    return "Hello, " + name;
  }
  // Arrow function
  const square = x => x * x;
  ```

- Functions can take arrays as arguments or return arrays.
- Array methods such as `.forEach()`, `.map()`, `.filter()`, and `.reduce()` expect a function to process or evaluate each array element.

Example: Using Functions with Arrays

```
const nums = [1, 2, 3];
const squares = nums.map(function(x) {
  return x * x;
});
console.log(squares); // Output: [1, 4, 9]
[^4][^6]
```

Common Array Methods that Use Functions:

Method	Purpose	Example usage
forEach()	Run a function for each element	nums.forEach(fn)
map()	Create a new array by mapping	nums.map(fn)
filter()	New array with elements passing	nums.filter(fn)
reduce()	Reduce values to a single result	nums.reduce(fn, acc)

Arrays and functions combined allow you to process, search, and transform data powerfully and concisely.

Javascript Constructors

What a constructor is

A function intended to create/initialize objects, usually called with new.

new does four things:

1. Creates a new empty object.
2. Sets its internal prototype to the constructor's prototype.
3. Binds this to that new object and runs the constructor body.
4. Returns the object (unless the constructor returns a different object explicitly).

Constructor Function

```
'use strict';
function Person(name) {
  // Guard so it works with or without `new`
  if (!new.target) return new Person(name);
  this.name = name;
}
Person.prototype.greet = function () {
  console.log(`Hi, I'm ${this.name}`);
};
const p = new Person('Ada');
p.greet(); // Hi, I'm Ada
```

Note:

- Don’t use arrow functions as constructors (they can’t be called with new).
- Put methods on FunctionName.prototype so they’re shared across instances.

Class Constructor

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
  static from(obj) {
    return new Person(obj.name);
  }
}

const p = new Person('Ada');
      p.greet();
```

Note:

- Class methods are on the prototype by default (shared).
- Classes are syntactic sugar over prototypes.

Pattern Matching Using Regular Expressions (RegEx):

What are Regular Expressions?

- A **Regular Expression (RegEx)** is a sequence of characters defining a search pattern used to match character combinations in strings.
- In JavaScript, RegEx are represented by RegExp objects or literals enclosed in forward slashes:

```
let pattern = /abc/;
```

Creating Regular Expressions

1. **Literal notation:**

```
let regex = /hello/i;
```

   - Matches the word "hello" case-insensitively (i flag).

2. **Constructor notation:**

```
let regex = new RegExp("hello", "i");
```

   - Useful for dynamic patterns.

Common Flags

Flag	Meaning
g	Global search (find all matches)
i	Case-insensitive search
m	Multiline search

Basic Pattern Examples

- `/abc/` — matches exact substring "abc".
- `/ab*c/` — matches a followed by zero or more b's then c.
- `/\d/` — matches any digit.
- `/[abc]/` — matches any one character 'a', 'b', or 'c'.
- `/[^abc]/` — matches any character except 'a', 'b', or 'c'.
- `/^Hello/` — match "Hello" at the start of a string.
- `/world$/` — match "world" at the end of a string.

Important Special Characters and Concepts

Character	Meaning
.	Any single character except newline
\d	Digit (0-9)
\w	Word character (a-z, A-Z, 0-9, _)
\s	Whitespace
\b	Word boundary
*	Zero or more repetitions
+	One or more repetitions
?	Zero or one repetition
()	Grouping & capturing
|	Alternation ("or" operator)

Methods to Use RegEx in JavaScript

Method	Description	Example
test()	Returns true if pattern matches a string, else false.	/abc/.test("abcde") // true
exec()	Returns array with match details or null.	/a(b)c/.exec("abc") // ["abc","b"]
match()	Calls RegExp on a string; returns array of matches or null.	"abc".match(/a./) // ["ab"]
matchAll()	Returns iterator over all matches, including groups.	
search()	Returns index of first match or -1 if none.	"abc".search(/b/) // 1

replace()	Replaces matched substring with a replacement string.	"abc".replace(/b/, "x") // "axc"
replaceAll()	Replaces all matches in string (when g flag used).	
split()	Splits a string by regex pattern into array of substrings.	

Example Usage

```
let text = "Hello, hello world!";

// Case-insensitive search for "hello"
let regex = /hello/gi;

// test method
console.log(regex.test(text)); // true

// match method returns all matches
console.log(text.match(regex)); // ["Hello", "hello"]

// replace all occurrences
console.log(text.replace(regex, "hi")); // "hi, hi world!"

// exec method returns capturing groups if any
let phone = "(123) 456-7890";
let pattern = /\((\d{3})\) \d{3}-\d{4}/;
console.log(pattern.exec(phone)); // ["(123) 456-7890", "123"]
```

Escaping Special Characters

- Use backslash \ to escape special characters.
  Example: To match a plus sign +, use /\+/.

Anchors

- ^ — Matches start of string or line.
- $ — Matches end of string or line.

Errors in Scripts

Errors in JavaScript scripts are problems that prevent the code from running as expected. They can occur during development or runtime and generally cause the program to stop executing further statements unless handled properly. Here's an overview of common types of errors and how they are managed in JavaScript:

Common Types of JavaScript Errors

1. **Syntax Errors**
    - Occur when code violates language syntax rules (e.g., missing parentheses, quotes, or braces).
    - These errors are detected at parse time before code execution.
    - Example: `console.log("Hello World);` causes a SyntaxError due to a missing closing quote.
    - Cannot be caught by try-catch since parsing fails before execution.
2. **Reference Errors**
    - Occur when referencing a variable that has not been declared.
    - Example: `console.log(x);` when `x` is undefined triggers a ReferenceError.
3. **Type Errors**
    - Occur when a value is used in an inappropriate way, such as calling a method on a value that doesn't support it.
    - Example: `let num = 5; num.toUpperCase();` throws a TypeError since a number does not have that method.

4. **Range Errors**
    - Occur when a value is outside an allowable range, e.g., creating an array with a negative length.
5. **URI Errors**
    - Result from incorrect use of URI functions (like `decodeURI`) when illegal characters are present.
6. **Custom Errors**
    - Users can create and throw their own errors using the `throw` statement to provide clearer messages.
7. **Internal Errors**
    - Errors like too much recursion or stack overflow in the JavaScript engine.

How Errors Affect Script Execution

- When an error is thrown and not caught, script execution halts at the point of the error.
- No code after the thrown error will run unless the error is handled.

Error Handling in JavaScript

JavaScript provides mechanisms to handle errors gracefully:

- **try...catch:**
  Wrap code that might throw an error inside `try`. If an error occurs, the `catch` block runs, allowing the program to continue.

```
try {
  let result = riskyFunction();
} catch (error) {
  console.error("An error occurred:", error.message);
}
```

- **finally:**
  An optional block that runs after try/catch whether or not an error occurred. Useful for cleanup actions.

- **throw:**
  Used to manually raise exceptions. Can throw strings, numbers, booleans, or custom error objects.
  Example:

```
if (!isValid(input)) {
  throw new Error("Invalid input");
}
```

Special Note About Script Errors

- A generic "Script Error" often appears in browser consoles when an error occurs in a script loaded from a different domain due to cross-origin restrictions.
- Browsers do not reveal detailed error info in such cases for security reasons unless proper CORS headers and attributes (`crossorigin="anonymous"`) are set.

Summary Table of Common JavaScript Errors

Error Type	Cause	Example
SyntaxError	Incorrect syntax (missing braces, quotes)	`console.log("Hello);`
ReferenceError	Using undeclared variables	`console.log(x);` when `x` undefined
TypeError	Incompatible operation or method call	`5.toUpperCase()`
RangeError	Value outside acceptable range	`Array(-1)`
URIError	Invalid use of URI functions	`decodeURI("%")`

Custom Error	Manually thrown error	`throw new Error("Oops!")`
Internal Error	Engine error like too much recursion	Excessive recursive calls

Using proper error handling with `try`, `catch`, `throw`, and optionally `finally` enables better debugging, smooth application execution, and a better user experience.

The JavaScript Execution Environment

The **JavaScript Execution Environment** consists mainly of two cooperating parts: the **JavaScript engine** and the **host environment**.

- The **JavaScript engine** is the core part that implements the ECMAScript specification. It takes JavaScript source code, parses it, compiles it (just-in-time in modern engines), and executes it. The engine manages memory allocation, maintains the call stack, and runs the execution contexts that represent running code blocks.
- The **host environment** provides additional APIs and interfaces outside the core language to interact with the system or platform where JavaScript runs. For example, in browsers, the host environment provides the DOM APIs for webpage interaction, timers, and network access APIs. Node.js, being another host environment, exposes file system, network, and other backend services to JavaScript code.

Key concepts inside the JavaScript engine within the execution environment include:

1. **Execution Context**: This is the abstraction representing the environment where JavaScript code runs. Every running script or function runs inside an execution context that keeps track of variables, functions, scope, and the value of `this`.
   There are two main types:
   - Global Execution Context: Created when the program starts.
   - Function Execution Context: Created each time a function is called.
2. **Execution Phases within an Execution Context**:

    - **Creation Phase**: The JavaScript engine scans the code, creates the variable environment, hoists variable and function declarations, creates the scope chain, and determines the value of `this`.
    - **Execution Phase**: The engine executes the code line by line, assigning values and running statements.
3. **Call Stack (Execution Stack)**: JavaScript uses a single-threaded call stack to manage which execution context is currently active. When a function is called, a new execution context is pushed onto the stack. When the function returns, its execution context is popped off.
4. **Memory Heap**: An area where objects, functions, and reference types are stored dynamically during execution.
5. **Web APIs (in browser environment)**: APIs provided by the host environment to allow asynchronous operations, DOM manipulation, events, timers, and more.
6. **Callback Queue and Event Loop**: JavaScript environment handles asynchronous callbacks through an event loop mechanism that pulls callbacks from the callback queue to the call stack when it is empty, facilitating non-blocking operations.

In summary, the JavaScript execution environment is a complex system combining the JavaScript engine that handles the language semantics and the host environment that provides additional capabilities specific to the platform. The execution model revolves around managing execution contexts dynamically using the call stack and memory heap, enabling synchronous and asynchronous execution of JavaScript code.

The Document Object Model

The **Document Object Model (DOM)** is a programming interface and representation of a web document (such as an HTML or XML page) that allows scripts and programs—primarily JavaScript—to access, manipulate, and change the content, structure, and style of the page dynamically.

Key Concepts of the DOM:

- The DOM represents a document as a **tree-like structure** of **nodes** and **objects**:

    - The root node is the entire document.
    - Child nodes represent HTML elements, attributes, and text content.
    - Each element is an object that can have properties (attributes) and methods.
- The DOM tree allows hierarchical navigation:
    - Nodes have parent, child, and sibling relationships.
    - Example: An HTML document with `<html>` as the root element, containing `<head>` and `<body>` elements, which in turn contain other elements like `<title>`, `<h1>`, and `<p>`.
- Nodes include:
    - **Element nodes**: Represent tags like `<div>`, `<p>`, etc.
    - **Text nodes**: Contain the text inside elements.
    - **Attribute nodes**: Represent attributes of elements, like `id`, `href`, etc.

How JavaScript Uses the DOM:

- JavaScript interacts with the web page content through the DOM.
- The global `document` object represents the webpage itself and provides numerous methods to:
    - Access elements by ID, class, tag, or CSS selectors (`getElementById()`, `querySelector()`, etc.).
    - Create, modify, or delete elements and attributes.
    - Change styles and content dynamically.
    - Attach event listeners for user interactions.

Example:

```
// Select an element by ID and change its text content
const heading = document.getElementById('myHeading');
heading.textContent = 'Hello, DOM!';
```

```
// Create new elements and add them to the page
const newPara = document.createElement('p');
newPara.textContent = 'This paragraph was added by JavaScript';
document.body.appendChild(newPara);
```

Why is the DOM Important?

- It bridges static HTML documents and dynamic, interactive web pages.
- It enables client-side scripting languages like JavaScript to manipulate page content and respond to user actions without reloading.
- The DOM is language- and platform-independent but is most commonly used with JavaScript in browsers.

Summary

- The DOM is a standardized **object model** representing the structure of web documents.
- It organizes documents into a hierarchical tree of nodes and objects.
- JavaScript accesses and manipulates the DOM to create dynamic web experiences.
- The `document` object is the entry point to the DOM in JavaScript environments.

Understanding the DOM is fundamental for front-end web development, enabling powerful control over web page content and interaction.

Element Access in JavaScript

Element Access in JavaScript to interact with the DOM can be done using several built-in methods provided by the `document` object. These methods allow you to select elements by their ID, class, tag name, or through CSS selectors.

Here are the main ways to access elements:

Method	Description	Returns	Example
`getElementById(id)`	Selects a single element with the specified unique ID.	Single Element or null	`document.getElementById("header")`
`getElementsByClassName(className)`	Selects all elements with the given class name.	HTMLCollection (live)	`document.getElementsByClassName("item")`
`getElementsByTagName(tagName)`	Selects all elements of the given tag name (e.g., "p", "div").	HTMLCollection (live)	`document.getElementsByTagName("li")`

querySelector(selector)	Selects the first element that matches a CSS selector.	Single Element or null	document.querySelector(".menu .active")
querySelectorAll(selector)	Selects all elements matching a CSS selector.	Static NodeList	document.querySelectorAll("ul > li")

Details and Usage:

- **getElementById** is the most straightforward way to find a single element by its unique id attribute. It returns null if no element matches.
- **getElementsByClassName** and **getElementsByTagName** return live collections of elements, meaning changes in the DOM affect the collection automatically.
- **querySelector** and **querySelectorAll** accept any valid CSS selector, making them very flexible. querySelector returns the first matching element, while querySelectorAll returns a static list of all matching elements.
- Example:

```
const header = document.getElementById("header");
const buttons = document.getElementsByClassName("btn");
const paragraphs = document.getElementsByTagName("p");
const firstActive = document.querySelector(".active");
```

```
const allItems = document.querySelectorAll(".list-item");
```

Knowing these methods allows you to efficiently access and manipulate HTML elements in your JavaScript code.

Events and Event Handling

Here is a comprehensive overview of **Events and Event Handling in JavaScript**:

What Are JavaScript Events?

- JavaScript events are **actions or occurrences** that happen in the web browser, such as user interactions (clicks, key presses), browser actions (page load, resize), or other DOM changes.
- Events allow you to make web pages interactive by responding to users or system-generated actions dynamically.

How JavaScript Event Handling Works

- **Event handling** means executing specific code in response to an event.
- You "listen" for an event on a DOM element, and when the event occurs, a function (event handler) runs.

Common Ways to Set Event Handlers

1. **Inline HTML Attributes** (Less recommended):

```
<button onclick="alert('Clicked!')">Click me</button>
```

2. **DOM Property Handlers**:

```
const btn = document.getElementById("btn");
btn.onclick = function() {
  alert("Clicked!");
};
```

3. **addEventListener() Method (Preferred)**:

```
btn.addEventListener("click", function() {
  alert("Clicked using addEventListener!");
});
```

- Allows multiple event listeners on the same element and event.
- Offers more control (e.g., event capturing vs bubbling).
- Allows efficient adding/removing of event handlers.

Common Event Types

- **Mouse events:** `click`, `dblclick`, `mouseover`, `mouseout`, `mousemove`, `mousedown`, `mouseup`
- **Keyboard events:** `keydown`, `keyup`, `keypress`
- **Form events:** `submit`, `change`, `input`, `focus`, `blur`
- **Window events:** `load`, `resize`, `scroll`
- **Touch events:** `touchstart`, `touchmove`, `touchend`

Event Object

- When an event handler is called, it receives an **event object** that contains details about the event, such as:
  - `event.target`: The element where the event originated.
  - `event.type`: The type of the event.
  - `event.preventDefault()`: Method to prevent the browser's default action (e.g., form submission reload).
  - `event.stopPropagation()`: Stops the event from bubbling up or capturing down the DOM tree.

Example: Button Click Event

```
<button id="myButton">Click me</button>
<script>
  const button = document.getElementById("myButton");
```

```
  button.addEventListener("click", function(event) {
    alert("Button clicked!");
  });
</script>
```

Example: Form Submit Handling With Preventing Default Action

```
<form id="myForm">
  <input type="text" id="username" />
  <input type="submit" value="Submit" />
</form>
<script>
  const form = document.getElementById("myForm");

  form.addEventListener("submit", function(event) {
    event.preventDefault(); // Stops page reload
    const username = document.getElementById("username").value;
    console.log("Username submitted:", username);
  });
</script>
```

Event Propagation: Bubbling and Capturing

- Events propagate through the DOM in two main phases:
    - **Capturing phase**: the event travels down from the root to the target.
    - **Bubbling phase**: the event bubbles back up from the target to the root.
- Using `addEventListener(type, listener, useCapture)` you can specify in which phase to listen:
    - `useCapture = true` listens in the capturing phase.
    - `useCapture = false` (default) listens in the bubbling phase.

Event Delegation

- Instead of attaching event listeners to many child elements, attach a single listener to a parent element and check `event.target` to handle events efficiently.
- Useful for dynamic content.

Summary

- JavaScript events enable interaction and dynamic behavior.
- Use `addEventListener()` for flexible and powerful event handling.
- Understand event object properties and methods to control event flow.
- Use event delegation for efficient event management on many elements.

Handling Events from Body Elements

Handling events from **body elements** in JavaScript involves adding event listeners or assigning event handlers directly to the `<body>` element. The `<body>` element supports many event types similar to other HTML elements, but also exposes several global event handlers related to window or document-level events.

Ways to Handle Events on the `<body>` Element

1. **Using HTML Attribute:**
   You can directly assign event handlers in the HTML tag of the body:

```
<body onload="init()" onclick="handleClick(event)">
```

This will run the `init()` function when the page loads, and `handleClick()` when the body is clicked.

2. **Using JavaScript DOM Property:**
   Assign handler functions to properties on `document.body`:

```
document.body.onload = function() {
  console.log("Page loaded");
};
```

```
document.body.onclick = function(event) {
  console.log("Body clicked", event.target);
};
```

This method supports one handler per event type; assigning again overwrites the previous.

3. **Using `addEventListener` (Recommended):**
   Add event listeners to the body for greater flexibility and multiple handlers per event:

```
document.body.addEventListener("click", function(event) {
  console.log("Body clicked:", event.target);
});

window.addEventListener("load", function() {
  console.log("Page fully loaded");
});
```

- Many events bubble up to the body, so attaching listeners here can handle events for many elements via event delegation.
- The body element supports events like `click`, `scroll`, `resize`, `load`, `unload`, `focus`, `blur`, and many more.

Common Events on `<body>` include:

- `onload` / `load`: Fired when the entire page (images, scripts, styles) has loaded.
- `onclick` / `click`: Fired on clicking anywhere inside body.
- `onscroll`: Fired when the page is scrolled.
- `onresize`: Often attached to `window` rather than body, but mirrors on body in some contexts.
- `onerror`: Captures errors that bubble up.
- `onfocus` / `onblur`: Focus events bubbling or specific to elements inside body.
- `ononline` / `onoffline`: Detect network connectivity changes.

Example: Handling click events on the body

```
<body>
  <button id="btn">Click me</button>
  <script>
    document.body.addEventListener("click", function(event) {
      console.log("Clicked element:", event.target.tagName);
    });
  </script>
</body>
```

Here, clicking anywhere inside the body (including the button) will trigger the body click event handler. The event object's `target` property indicates which element was actually clicked.

Why Handle Events on `<body>`?

- **Event delegation:** Instead of adding listeners to every element, listen on the body and use `event.target` to determine which element triggered the event.
- **Global page events:** Events like page load, network status, or errors can be captured at the body or window level.
- **Simplify event management:** One listener can serve many elements, especially for dynamic content.

Summary Table of `<body>` Event Handling Methods

Method	Description	Example
Inline HTML attribute	Add event handler inside `<body ...>` tag	`<body onload="init()">`
Direct DOM property	Assign function to `document.body.event`	`document.body.onclick = handler`
`addEventListener`	Add multiple listeners flexibly	`document.body.addEventListener("click", handler)`

Important Notes from HTML5 specification

- <body> mirrors many event handlers of the window object for convenience.
- Some events bubble from descendants and can be handled at body.
- Events attached on body with addEventListener get called during bubbling phase.

Handling Events from Button Elements

To handle events from **button elements** in JavaScript, the recommended approach is to use the addEventListener() method on the button element. This method attaches an event handler function that will execute when the specified event occurs, such as a "click" event.

How to Handle Events for a Button Element

1. **Select the button element**, for example by its id, class, or query selector.
2. **Use addEventListener()** to bind an event type (like "click") to a handler function.

Syntax

```
buttonElement.addEventListener(eventType, eventHandlerFunction, useCapture);
```

- eventType: The type of event as a string, for example, "click".
- eventHandlerFunction: The function to run when the event occurs.
- useCapture (optional): Boolean indicating if the event should be captured during the capturing phase (true) or bubbling phase (false). Default is false.

Example: Simple Click Event Handler on a Button

```
<button id="myButton">Click me</button>
<script>
  // Select the button element by its ID
  const button = document.getElementById("myButton");
```

```
  // Add click event listener
  button.addEventListener("click", function() {
    alert("Button clicked!");
  });
</script>
```

Example: Named Function as Event Handler

```
function handleClick() {
  console.log("Button was clicked!");
}

const btn = document.querySelector("button");
btn.addEventListener("click", handleClick);
```

Adding Multiple Event Handlers to a Button

You can attach multiple event listeners of the same or different event types to a single button.

```
button.addEventListener("click", () => {
  console.log("Click handler 1");
});

button.addEventListener("click", () => {
  console.log("Click handler 2");
});

button.addEventListener("mouseenter", () => {
  console.log("Mouse entered button");
});
```

Benefits of Using `addEventListener()` for Buttons

- Allows multiple event listeners on the same element without overwriting.
- Separates JavaScript from HTML markup improving organization.
- Provides flexibility with event phases (capturing/bubbling).
- Enables easy removal of listeners via `removeEventListener()` if needed.

These are the standard and preferred methods to handle button events in modern JavaScript applications.

Handling Events from Text Box and Password Elements

Handling events from **text box** (`<input type="text">`) and **password** (`<input type="password">`) elements in JavaScript primarily involves listening to user interactions such as typing, focusing, changing, and submitting data. The most commonly used events for text and password inputs include:

Common Events for Text Box and Password Input Elements

- **`input event`**: Fires immediately whenever the value changes, such as when the user types or deletes a character. Useful for real-time validation or feedback.
- **`change event`**: Fires when the element loses focus after its value has changed. Useful for capturing final input after editing.
- **`focus event`**: Fires when the input gains focus (user clicks or tabs into the field).
- **`blur event`**: Fires when the input loses focus.
- **`keydown, keyup, keypress events`**: Fire on keyboard key presses and releases, useful for custom key handling or restrictions.

How to Attach Event Handlers

The recommended way is using `addEventListener()` on the input element:

```
const textBox = document.getElementById("myText");
const passwordBox = document.getElementById("myPassword");

// Input event for live input processing
```

```
textBox.addEventListener("input", function(event) {
  console.log("Current input: ", event.target.value);
});

// Change event for value committed after editing
passwordBox.addEventListener("change", function(event) {
  console.log("Password changed to: ", event.target.value);
});

// Focus event example
textBox.addEventListener("focus", function() {
  console.log("Text box is focused");
});

// Blur event example
passwordBox.addEventListener("blur", function() {
  console.log("Password box lost focus");
});
```

Example: Basic HTML + JavaScript

```
<input type="text" id="myText" placeholder="Enter username" />
<input type="password" id="myPassword" placeholder="Enter password" />

<script>
  document.getElementById("myText").addEventListener("input", function(e) {
    console.log("Text input: ", e.target.value);
  });

  document.getElementById("myPassword").addEventListener("change", function(e) {
    console.log("Password input changed.");
  });
</script>
```

Additional Notes:

- The **input** event is preferred for live validation because it triggers immediately as the user types, while **change** fires only after the field loses focus following a change.
- Keyboard events (keydown, keyup, keypress) provide lower-level control for specialized behavior, but input and change are sufficient for most use cases.
- You can prevent form submission behavior on form elements via event.preventDefault() during the submit event to validate inputs before sending.

These event handlers enable efficient and responsive handling of user input in text and password fields.

The DOM 2 Event Model

The **DOM Level 2 Event Model** is a standardized system designed to handle events programmatically within the Document Object Model. It aimed to create a generic, flexible event framework that supports event handler registration, event flow through the DOM tree, and contextual event information.

Key Features and Concepts of the DOM 2 Event Model:

- **Event Flow (Three Phases):**
    a. **Capturing Phase:** The event propagates from the root of the document down to the target element. Ancestors of the target can intercept the event in this phase.
    b. **Target Phase:** The event reaches the event target element itself, triggering its handlers.
    c. **Bubbling Phase:** After reaching the target, the event bubbles back up the DOM tree to the ancestors, invoking their handlers if registered for the bubbling phase.
- **Event Listeners:**
    a. Multiple event listeners can be registered for the same event type on the same element, allowing more flexible event management.

b. Event listeners can specify whether they should be triggered during the capturing phase or bubbling phase by the boolean `useCapture` parameter in `addEventListener(type, listener, useCapture)`.

- **Event Object:**

a. Provides detailed information about the event, including the event type, target element (`event.target`), current target (`event.currentTarget`), and which phase of the event flow is currently executing (`event.eventPhase`).

b. Methods like `stopPropagation()` and `preventDefault()` allow stopping further propagation of the event and preventing default browser actions, respectively.

- **Standard Event Sets:**

a. The model defines standard sets of events for user interface control, logical UI events, and document mutation events, intending to be interoperable across browsers.

- **Compatibility Goal:**

a. The model attempts to provide a common subset of behavior consistent with previous event models used in older browsers (like IE 4.0 and Netscape Navigator 4.0), promoting interoperability.

Main Interfaces and Methods:

- `addEventListener(type, listener, useCapture)`: Registers an event listener on a DOM node.
- `removeEventListener(type, listener, useCapture)`: Removes a previously registered event listener.
- Event propagation through three phases is controlled by the parameters and event flow order.

Summary of Event Flow Phases:

Phase	Description	useCapture Parameter
Capturing	Event goes down from root to target	`true`
Target	Event reached the event target itself	N/A (default phase)

Bubbling	Event bubbles up from target to root	false

Example of Registering Event Listeners:

```
element.addEventListener("click", handler, true);  // Capturing phase
element.addEventListener("click", handler, false); // Bubbling phase (default)
```

Advantages over DOM Level 0 and 1 Models:

- Supports multiple handlers per event per element.
- Separate capturing and bubbling phases provide greater control over event handling order.
- Standardized event object providing consistent properties and methods across browsers.

This model is the foundation for modern DOM event handling and is supported by all major browsers today.